

Содержание

1. Что такое ORM?
2. Как работает ORM?
3. Настройка SQLAlchemy
4. Что такое модель и как ее создать?
5. Что такое сессия (session)?
6. Добавление объектов в БД
7. Простой SELECT запрос

1. Что такое ORM?

Мы уже решали задачи с реляционными базами данных. Однако пока мы работали с данными исключительно с помощью запросов на языке SQL, которые составляли самостоятельно. Для небольших проектов это вполне оправдано и достаточно часто используется в реальной жизни.

Согласитесь: очень хочется перейти на более высокий уровень работы с данными. Представьте, что в вашем распоряжении есть объект, который связан с базой данных. Этот объект берет на себя всю работу по организации общения с данными. Вам лишь остается давать ему команды: получить данные, отфильтровать их по заданному условию, записать данные и т. д., а преобразование команд в SQL-запросы — это уже забота объекта. Работать с данными как с объектами чрезвычайно удобно

К сожалению, написать универсальный модуль, который преобразовывает результат SQL-запроса в объекты и наоборот, достаточно нетривиальная задача. Хорошо, что она уже решена за нас.

В больших приложениях достаточно часто используется технология ORM (Object-Relational Mapping — объектно-реляционное отображение) — прослойка, позволяющая работать с базой данных через объекты языка. Кроме того, большинство ORM позволяют генерировать скрипты миграции базы данных для поддержания версииности (отдаленно можно сравнить с git, но

для баз данных), а также предоставляют разработчику еще немало другой полезной функциональности. Мы будем использовать библиотеку sqlalchemy.

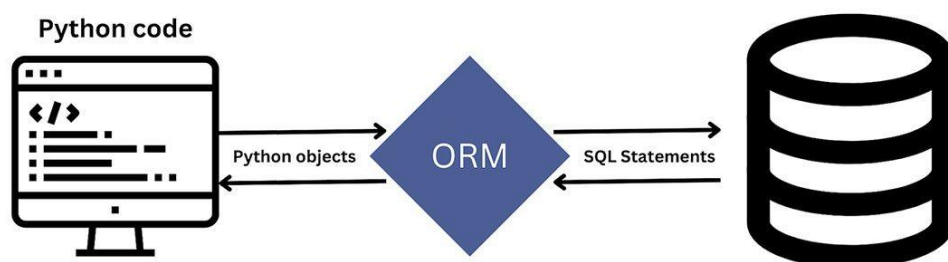
```
# Без ORM (сложно и много кода)
cursor.execute("INSERT INTO cats (name, age) VALUES ('Мурзик', 3)")

# С ORM (просто и понятно)
new_cat = Cat(name="Мурзик", age=3)
session.add(new_cat)
```

2. Как работает ORM?

Представьте, что ORM — это переводчик между вашим приложением и базой данных:

- Объекты Python → письма, которые вы пишете на русском
- ORM → переводчик, который переводит их на английский (SQL)
- Драйвер БД → почтовая служба, которая доставляет письма
- Ответ БД → письмо на английском, которое ORM переводит обратно на русский



3. Настройка SQLAlchemy

Шаг 1: Установка

```
pip install sqlalchemy
```

Шаг 2: Подключаем компоненты

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.ext.declarative import declarative_base

# Движок — это как "ключ зажигания" для работы с БД
engine = create_engine('sqlite:///school.db')

# Сессии (подробнее в следующем разделе)
Session = sessionmaker(bind=engine)
session = Session()

# BaseModel — шаблон для всех моделей
BaseModel = declarative_base()
```

В функции `create_engine()` указана строка, которая состоит из типа базы данных, адреса до базы данных и параметров подключения. Её мы передаем SQLAlchemy для того, чтобы она выбрала правильный движок работы с базой данных (переменная `engine`). В нашем случае это будет движок для работы с SQLite базами данных.

Если в функцию `create_engine()` передать параметр `echo` со значением `True`, в консоль будут выводиться все SQL-запросы, которые сделает SQLAlchemy, что очень удобно для отладки.

4. Что такое модель и как ее создать?

Модель — это инструкция для ORM, как создать таблицу в БД. Она описывает структуру таблицы, включая названия колонок, их типы и дополнительные параметры.

Колонки в SQLAlchemy создаются с помощью функции `Column`. Она принимает несколько параметров, которые определяют свойства колонки.

Тип данных для `Column` (первый аргумент). Определяет, какие данные могут храниться в колонке. Примеры типов:

- Integer — целое число.
- String(length) — строка с ограниченной длиной (например, String(50)).
- Text — строка без ограничения длины.
- Boolean — логическое значение (True/False).
- Float — число с плавающей точкой.
- DateTime — дата и время.
- Date — только дата.
- Time — только время.
- JSON — данные в формате JSON.

Параметры Column (указываются после типа данных):

- primary_key (По умолчанию False) - Если True, колонка становится первичным ключом (уникальный идентификатор строки).
- nullable (По умолчанию True) - Если False, колонка не может содержать значение NULL (обязательное поле).
- unique (По умолчанию False) - Если True, все значения в колонке должны быть уникальными.
- default (Например, default=0 или default="unknown") - Устанавливает значение по умолчанию для колонки, если значение не указано явно.
- index (По умолчанию False) - Если True, создается индекс для колонки, что ускоряет поиск по ней.
- autoincrement (По умолчанию True для колонок с типом Integer, если они являются первичными ключами.) - Если True, значение колонки автоматически увеличивается при добавлении новой строки (обычно используется для первичных ключей).
- server_default - Устанавливает значение по умолчанию на уровне базы данных (например, server_default="CURRENT_TIMESTAMP" для даты и времени).

- `onupdate` - Устанавливает значение, которое автоматически обновляется при изменении строки (например, `onupdate="CURRENT_TIMESTAMP"` для даты и времени).
- `name` (Например, `name="user_name"`) - Позволяет указать имя колонки в базе данных, если оно отличается от имени атрибута в модели.
- `comment` (Например, `comment="Имя пользователя"`) - Добавляет комментарий к колонке в базе данных.
- `foreign_key` (Например, `foreign_key="users.id"`) - Указывает, что колонка является внешним ключом, ссылающимся на другую таблицу.
- `doc` (Например, `doc="Имя пользователя, обязательно для заполнения"`) - Добавляет документацию к колонке (не влияет на базу данных, используется только в коде).

Пример модели «Книга»:

```
from sqlalchemy import Column, Integer, String

class Book(BaseModel):
    __tablename__ = 'book' # Название таблицы

    # Колонки создаются через Column
    id = Column(Integer, primary_key=True) # Главный ключ
    title = Column(String(100)) # Строка длиной 100 символов
    pages = Column(Integer) # Число

    def __repr__(self):
        return f"<Книга: {self.title}, {self.pages} стр.>"

# .... тут может быть описано сразу много моделей

# Создаем таблицы в БД (если их еще нет в ней)
# (Создаются все модели, который наследуются от BaseModel)
BaseModel.metadata.create_all(engine)
```

5. Что такое сессия (session)?

Сессия (Session) — это объект, через который происходит общение с БД. За одну сессию может быть выполнено сразу несколько запросов.

Как блокнот для записей:

- Все изменения (добавление, удаление) сначала сохраняются в сессии
- Только после `commit()` они отправляются в БД

Зачем нужна?

- Можно отменить еще несохраненные изменения (`rollback()`), если что-то пошло не так
- Управляет подключениями к БД автоматически

6. Добавление объектов в БД

```
# 1. Создаем объект
new_book = Book(title="Гарри Поттер", pages=400)

# 2. Добавляем в сессию (но пока не в БД!)
session.add(new_book)

# 3. Сохраняем изменения в БД
session.commit()

# 4. Закрываем сессию
session.close()
```

7. Простой SELECT запрос

Получить все объекты:

```
all_books = session.query(Book).all() # all() — все записи

for book in all_books:
    print(f"{book.title} ({book.pages})")
```

Поиск по условию:

```
# Найти книги, в которых меньше 200 страниц
small_books = session.query(Book).filter(Book.pages < 200).all()

# Найти первую книгу со словом minecraft в названии
book = session.query(Book).filter(Book.title.like("%minecraft%")).first()
```